

Objectifs

À l'issue de cette section, l'étudiant devra être capable de :
— ok

La programmation dynamique est une méthode de résolution de problèmes d'optimisation. Son principe général est de décomposer un problème en sous-problèmes, de résoudre chacun des sous-problèmes pour construire la solution au problème initial. Son intérêt est d'éviter les calculs redondants susceptibles d'apparaître dans les sous-problèmes qui ne sont plus nécessairement indépendants, réduisant significativement la complexité temporelle du problème général.

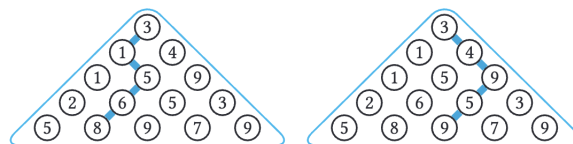
1 Pyramide d'entiers

On considère une pyramide d'entiers naturels de hauteur n , entier naturel non nul. Un premier objectif est de déterminer la plus grande somme qu'il est possible de calculer en parcourant la pyramide de haut en bas, les seuls parcours autorisés étant ceux qui font passer d'un entier à l'un des deux entiers situés immédiatement en dessous. Un tel parcours est appelé chemin dans la pyramide. La plus grande somme calculée constitue la solution de valeur optimale au problème.

Mais rien n'indique comment elle est calculée. Un second objectif est donc de déterminer les entiers de la pyramide qui ont permis la détermination de cette valeur optimale. Il s'agit là de construire la solution optimale.

Exemple

Voici une pyramide d'entiers pour laquelle un seul chemin mène à une plus grande somme égale à 30.



Approche exhaustive.

L'approche exhaustive traite tous les chemins possibles et détermine celui de valeur optimale. Partant du sommet de la pyramide, deux chemins mènent aux entiers situés sur le niveau immédiatement inférieur, deux nouveaux chemins partent de chacun de ces entiers vers le niveau encore inférieur, et ainsi de suite.

Le nombre de chemins menant à la profondeur p dans la pyramide est donc le double du nombre de chemins ayant mené à la profondeur précédente $p - 1$. Le dernier niveau

de la pyramide de hauteur n est situé à la profondeur $n - 1$. Avec deux chemins menant à la profondeur 1, on aboutit à 2^{n-1} chemins en tout dans la pyramide. Un algorithme exhaustif n'est donc pas pertinent dès que la hauteur de la pyramide dépasse quelques dizaines de niveaux.

Approche gloutonne.

Dans certaines situations, une approche gloutonne peut mener à une solution convenable, avec des complexités temporelles raisonnables.

Dans le cas présent, cette approche maximiserait la somme à chaque étape et, dans l'exemple de la pyramide de la figure précédente, fournirait un résultat correct. Mais ce n'est pas toujours le cas.

En outre, il ne s'agit pas ici de trouver une solution convenable mais bien de trouver une solution optimale, ce que l'approche gloutonne est incapable de garantir à tous les coups pour ce problème.

1.1 Approche récursive.

Une approche récursive décompose le calcul de la plus grande somme en sous-calculs. Ex- trayons deux sous-pyramides comme sur la figure 1, dont les sommets sont les entiers situés immédiatement sous le sommet de la pyramide.



FIGURE 1 – Sous-pyramides gauche et droite de la pyramide

Si, par deux calculs préalables, la plus grande somme de chacune des deux sous-pyramides est connue, celle de la pyramide initiale s'en déduit.

Pour écrire cette somme, introduisons les notations suivantes.

Pour des entiers $i \in [0, n - 1]$ et $j \in [0, i]$, on note $a_{i,j}$ le j -ième élément de la i -ième ligne de la pyramide.

On note $s_{i,j}$ la plus grande somme associée à la sous-pyramide de sommet $a_{i,j}$. On a alors

$$\forall i \in [0, n - 1] \quad \forall j \in [0, i] \quad s_{i,j} = a_{i,j} + \max(s_{i+1,j}, s_{i+1,j+1})$$

en posant $\forall j \in [0, n] \quad s_{n,j} = 0$.

Avec les choix de notations adoptés, la relation de récurrence s'écrit avec des indices croissants.

Une telle relation se prête à un codage récursif simple. Les entiers de la pyramide sont stockés dans un tableau de tableaux, chaque sous-tableau étant associé à une ligne de la pyramide. En OCaml, le tableau associé à la pyramide de la figure 1 est le suivant :

`[[3]; [1; 4]; [1; 5; 9]; [2; 6; 5; 3]; [5; 8; 9; 7; 9]]`

La fonction `max_sum_rec` du programme suivant calcule la plus grande somme dans un tableau associé à une pyramide.

Programme pour la plus grande somme avec approche récursive

```
let max_sum_rec (a: int array array) : int =
  let rec s i j = (* 0 <= i <= j <= |a| *)
    if i = Array.length a then 0
    else a.(i).(j) + max (s (i+1) j) (s (i+1) (j+1))
  in
  s 0 0
```

Bien que simple et concis, ce code est malheureusement inutilisable pour des tableaux dont la taille dépasse la centaine d'éléments, les appels récursifs menant à une complexité temporelle exponentielle. Cette dernière est directement liée au nombre d'appels récursifs.

Pour un entier i fixé de $[0, n - 1]$, notons c_i le nombre d'appels récursifs pour calculer la plus grande somme associée à la pyramide de sommet $a_{i,j}$, avec $j \in [0, i]$. On peut remarquer que ce nombre ne dépend pas de j mais seulement de la hauteur d'une pyramide, donnée ici par $n - i$. En effet, les calculs de $s_{i+1,j}$ et de $s_{i+1,j+1}$ comportent le même nombre d'appels récursifs.

La lecture du code mène alors à la relation de récurrence $\forall i \in [0, n - 1] \quad c_i = 2c_{i+1}$

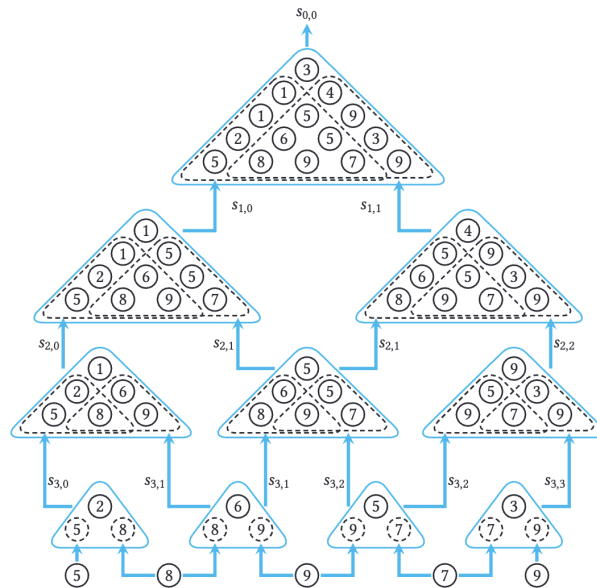
Sachant qu'un seul appel récursif suffit pour déterminer les quantités $s_{n-1,j}$, il vient $c_0 = 2^n$.

La complexité temporelle est donc en $\Theta(2^n)$.

Notons qu'il n'y a pas de risque de débordement de pile, car la mémoire sera remplie par la pyramide avant même que la valeur de n ne soit susceptible de provoquer un trop grand nombre d'appels récursifs imbriqués.

Une observation détaillée des calculs effectués lors des appels récursifs permet d'identifier la cause de la complexité temporelle exponentielle : des calculs sont faits plusieurs fois.

La figure suivante illustre cette observation.



Le calcul récursif de la plus grande somme est donné par :

$$s_{0,0} = 3 + \max(s_{1,0}, s_{1,1})$$

Si on développe les appels récursifs liés aux calculs de $s_{1,0}$ et de $s_{1,1}$, on obtient la relation suivante dans laquelle $s_{2,1}$ apparaît deux fois, signifiant que cette quantité est calculée deux fois.

$$s_{0,0} = 3 + \max(1 + \max(s_{2,0}, s_{2,1}), 4 + \max(s_{2,1}, s_{2,2}))$$

En développant encore un niveau d'appels récursifs, il apparaît que $s_{3,1}$ et $s_{3,2}$ sont calculés deux fois également.

$$s_{0,0} = 3 + \max(1 + \max(1 + \max(s_{3,0}, s_{3,1}), 5 + \max(s_{3,1}, s_{3,2})), 4 + \max(5 + \max(s_{3,1}, s_{3,2}), 9 + \max(s_{3,2}, s_{3,3})))$$

On pourrait poursuivre cette analyse jusqu'aux cas de terminaison mais ces premiers calculs, appuyés par la figure suivante, illustrent clairement les redondances calculatoires de l'approche récursive, justifiant par la même occasion les mauvaises complexités. Se pose alors la question de savoir par quel moyen éviter la répétition des calculs.

1.2 Mémoïsation.

Si, au fur et à mesure que des calculs sont faits, leurs résultats sont stockés en mémoire, il suffit alors de les récupérer chaque fois que cela est nécessaire plutôt que de les refaire.

Cette technique, qui porte le nom de mémoïsation, est simple à mettre en œuvre dès qu'on dispose d'une formulation récursive du problème.

L'idée est que,

- si un calcul n'a pas été fait, un cas de base ou un appel récursif doit le permettre et le résultat obtenu est conservé en mémoire, dans une structure adaptée
- si un calcul a déjà été fait, il suffit de lire l'information dans cette structure pour simplement récupérer le résultat, éliminant de fait tout calcul redondant.

Par souci d'efficacité, la structure de données qui stocke les résultats doit idéalement permettre des ajouts et des modifications en temps constant, comme par exemple un tableau ou une table de hachage.

Le programme suivante illustre la mise en œuvre de cette technique de mémoïsation pour calculer la plus grande somme.

Mémoïsation

```
let max_sum_memo (a: int array array) : int =
  let memo = Hashtbl.create 16 in
  let rec s i j = (* 0 <= j <= i <= |a| *)
    try
      Hashtbl.find memo (i, j)
    with Not_found ->
      let v =
```

```

        if i = Array.length a then 0
        else a.(i).(j) + max (s (i+1) j) (s (i+1) (j+1)) in
    Hashtbl.add memo (i, j) v;
  v
in
s 0 0

```

Ce code stocke les valeurs calculées dans un dictionnaire dont les clés sont les couples (i, j) d'entiers associés aux indices de $s_{i,j}$. Les appels récursifs sont limités aux stricts calculs de nouvelles valeurs non encore présentes dans le dictionnaire. Au début des calculs, le dictionnaire est vide et commence par se remplir progressivement dès les premiers calculs. Puis les résultats des appels récursifs se réduisent assez vite à de simples lectures des entiers stockés dans le dictionnaire en vue de produire de nouvelles valeurs si nécessaire.

Pour calculer la complexité de cette solution utilisant la mémoïsation, il faut constater que, pour tous $0 \leq j \leq i \leq n$, la fonction s est appelée sur i et j au moins une fois et au plus deux fois.

- Pour $i = j = 0$, il s'agit de l'appel initial.
- Pour toutes autres valeurs de i et j , il y a une ou deux valeurs au-dessus dans la pyramide. Pour chacune, seul le premier appel occasionne à un appel sur (i, j) car la mémoïsation interrompt la récursion dès la deuxième fois.

En conclusion, toutes les valeurs de la pyramide sont visitées par la fonction s mais au plus deux fois chacune. Comme le corps de la fonction s s'exécute en temps constant (hors appels récursifs), la complexité temporelle est donc en $\Theta(n^2)$. C'est optimal, car on est obligé d'examiner tous les éléments de la pyramide pour répondre à la question.

La complexité spatiale est également en $\Theta(n^2)$ car la table stocke au final tous les $s_{i,j}$ pour $0 \leq j \leq i \leq n$.

Nous allons maintenant voir que l'on peut améliorer ce dernier point.

1.3 Calcul de bas en haut.

L'approche précédente optimise les calculs dans le sens où elle évite les redondances calculatoires. C'est là une caractéristique essentielle de la programmation dynamique qui résout un problème en s'aidant de résultats de sous-problèmes répétés. On parle de *recouvrement* du problème par des sous-problèmes. Quand ces derniers interviennent plusieurs fois dans la construction de la solution au problème général, la programmation dynamique prend tout son sens.

Toutefois, si la solution par mémoïsation constitue une réponse satisfaisante au problème, la spécificité de ce dernier autorise parfois une amélioration de l'algorithme de résolution. Le programme de mémoïsation suit la logique de l'approche récursive dans le sens où les calculs sont menés du haut vers le bas (en anglais *top-down*). On cherche à calculer $s_{0,0}$ en appelant $s_{1,0}$ et $s_{1,1}$. À leur tour, ces quantités appellent $s_{2,0}$, $s_{2,1}$, $s_{2,2}$ et ainsi de suite. Pourtant le calcul peut se faire également du bas vers le haut (en anglais *bottom-up*).

C'est d'ailleurs l'idée sous-jacente à la programmation dynamique : construire une solution générale à partir de solutions particulières.

Cette approche se traduit généralement par un code itératif.

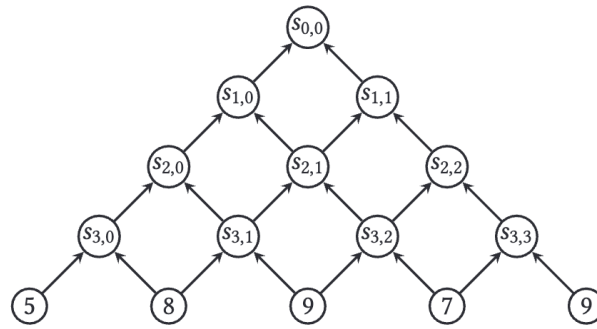


FIGURE 2 – Dépendances entre les valeurs calculées

Le programme suivante la réalise avec un tableau bidimensionnel s de taille $(n+1) \times (n+1)$, qui est progressivement rempli de bas en haut, à l'aide d'une double boucle, en suivant la relation de récurrence.

Programme pour la plus grande somme avec programmation dynamique de bas en haut

```
let max_sum_dp (a: int array array) : int =
  let n = Array.length a in
  let s = Array.make_matrix (n+1) (n+1) 0 in
  for i = n - 1 downto 0 do
    for j = 0 to i do
      s.(i).(j) <- a.(i).(j) + max s.(i+1).(j) s.(i+1).(j+1)
    done
  done;
  s.(0).(0)
```

La complexité en temps est trivialement $\Theta(n^2)$, car on a deux boucles imbriquées et un calcul en temps constant à chaque itération.

La complexité en espace est également $\Theta(n^2)$. C'est la place occupée par la matrice s .

1.4 Construction d'une solution optimale.

Les solutions précédentes déterminent la valeur optimale associée au problème, à savoir la plus grande somme dans le cas présent. Mais on peut souhaiter connaître les entiers de la pyramide qui permettent ce calcul. Il s'agit alors de construire la solution optimale. Le programme suivante est une

très légère modification de la fonction `max_sum_dp` qui conserve, dans un tableau `sol`, les listes des indices qui contribuent aux différentes sommes. Plus précisément, on a l'invariant

$$\forall k, s[k] = \sum_{(i,j) \in sol[k]} a_{i,j}.$$

Programme pour la plus grande somme

```

let max_sum_sol (a: int array array) : int * (int * int) list =
  let n = Array.length a in
  let s = Array.make (n+1) 0 in (* 0 <= i <= n *)
  let sol = Array.make (n+1) [] in
  for i = n - 1 downto 0 do
    for j = 0 to i do
      sol.(j) <- (i, j) ::
        if s.(j) > s.(j+1) then sol.(j) else sol.(j+1);
      s.(j) <- a.(i).(j) + max s.(j) s.(j+1)
    done
  done;
  (s.(0), sol.(0))

```

Comme on le voit, le code met à jour les deux tableaux s et sol en même temps et de manière cohérente. Au final, on renvoie la plus grande somme ainsi que la liste des indices qui y contribuent. Notons que si plusieurs solutions existent, la fonction `max_sum_sol` ne renvoie que l'une d'elles.

La complexité en temps est toujours $\Theta(n^2)$ mais la complexité en espace augmente, car l'ensemble des listes contenues dans le tableau sol occupe un espace $O(n^2)$ dans le pire des cas.

2 Rendu de monnaie

Le problème du rendu de monnaie a été présenté dans les algorithmes gloutons. On a alors constaté que l'approche gloutonne ne garantissait pas la meilleure solution à un problème d'optimisation mais seulement un optimum relatif. La programmation dynamique permet en revanche d'obtenir une solution optimale au problème du rendu de monnaie.

Rappelons que l'objectif est de rendre une somme d'argent s avec le moins de billets et de pièces possibles. On suppose donné un système monétaire S_n défini par un ensemble de n billets et pièces de monnaie de valeurs entières $v_1 < v_2 < \dots < v_n$, avec $v_1 = 1$. Notons $m_i(s)$ le nombre minimal de billets et de pièces utilisés pour rendre la somme s avec $S_i = \{v_1, v_2, \dots, v_i\}$, pour un entier $1 \leq i \leq n$. De manière évidente, on a tout d'abord les résultats suivants.

- Si $s = 0$, aucune somme n'est à rendre et $m_i(0) = 0$.
- Si $s = 1$, seule la pièce de valeur $v_1 = 1$ convient et $m_i(1) = 1$.
- Si $n = 1$, le système se réduit à la pièce de valeur $v_1 = 1$ et $m_1(s) = s \times 1 = s$.

Pour $i > 1$ et $s > 1$, puisque les valeurs $v_1, v_2, \dots, v_i$ sont croissantes, une somme s peut être rendue avec la pièce v_i si $v_i \leq s$. Ce qui mène aux deux résultats suivants.

- Si $v_i > s$, la pièce v_i ne peut jamais être utilisée et :

$$m_i(s) = m_{i-1}(s).$$

- Si $v_i \leq s$, deux cas sont envisageables.
 - Si l'utilisation de la pièce v_i minimise le nombre de pièces rendues, la somme restant à rendre est $s - v_i$ de sorte que $m_i(s) = 1 + m_i(s - v_i)$. Le 1 traduit

l'utilisation de la pièce v_i . Le terme $m_i(s - v_i)$ désigne le nombre minimal de pièces utilisées pour rendre la somme $s - v_i$ avec S_i .

- Si l'utilisation de la pièce v_i ne minimise pas le nombre de pièces rendues, c'est que seules les autres pièces doivent être utilisées, ce qu'on peut traduire par $m_i(s) = m_{i-1}(s)$.

Ces deux situations peuvent se résumer par la relation de récurrence suivante :

$$m_i(s) = \min(1 + m_i(s - v_i), m_{i-1}(s)).$$

Une telle formulation mathématique du problème mène naturellement à l'écriture d'un programme récursif. Mais la complexité exponentielle de cette solution la rend inutilisable. C'est tout à fait comparable à ce que nous avons vu avec la pyramide d'entiers.

Comme pour la pyramide d'entiers, on peut appliquer le principe de mémorisation, en stockant les valeurs $m_i(s)$ dans une matrice ou dans une table de hachage. La complexité temporelle et spatiale est alors $\Theta(ns)$. De même, on peut appliquer le principe de programmation dynamique de bas en haut. Le programme 9.16 en contient une réalisation en C. Il alloue une matrice m de taille $n \times (s + 1)$, qui est ensuite remplie pour des valeurs croissantes de i et des valeurs croissantes de la somme. L'ordre est important, car on utilise des valeurs précédemment calculées, plus à gauche sur la même ligne ou sur la ligne au-dessus.

Rendu de monnaie avec programmation dynamique

```
int dp_change(int coins[], int n, int s) {
    assert(n > 0 && coins[0] == 1 && s >= 0);
    int **m = calloc(n, sizeof(int*));
    for (int i = 0; i < n; i++) m[i] = calloc(s + 1, sizeof(int));
    // m[i][t] = solution pour une somme t avec les pièces 0, ..., i
    for (int t = 1; t <= s; t++)
        m[0][t] = t;
    for (int i = 1; i < n; i++)
        for (int t = 1; t <= s; t++) {
            m[i][t] = m[i-1][t]; // sans coins[i]
            int d = t - coins[i];
            if (d >= 0 && m[i][d] < m[i][t])
                m[i][t] = 1 + m[i][d]; // avec
        }
    int r = m[n-1][s];
    for (int i = 0; i < n; i++) free(m[i]);
    free(m);
    return r;
}
```

La complexité temporelle et spatiale est clairement $\Theta(ns)$. Comme pour la pyramide d'entiers, on peut faire baisser la complexité spatiale en ne conservant qu'une seule ligne de la matrice m .